AI STUDY BUDDY

A Cloud-Native AI Powered Learning Platform
using DevOps and Cloud Technologies

Submitted By

RAHUL TYAGI
12319156

Technologies Used

Python · Streamlit · Docker · Kubernetes · Jenkins
Terraform · AWS · Prometheus · Grafana · GitHub

GitHub Link: https://github.com/the-rahul-tyagi/AiStudyBuddyPro

Submitted To

Ms. Arshiya

Academic Year 2025 – 2026

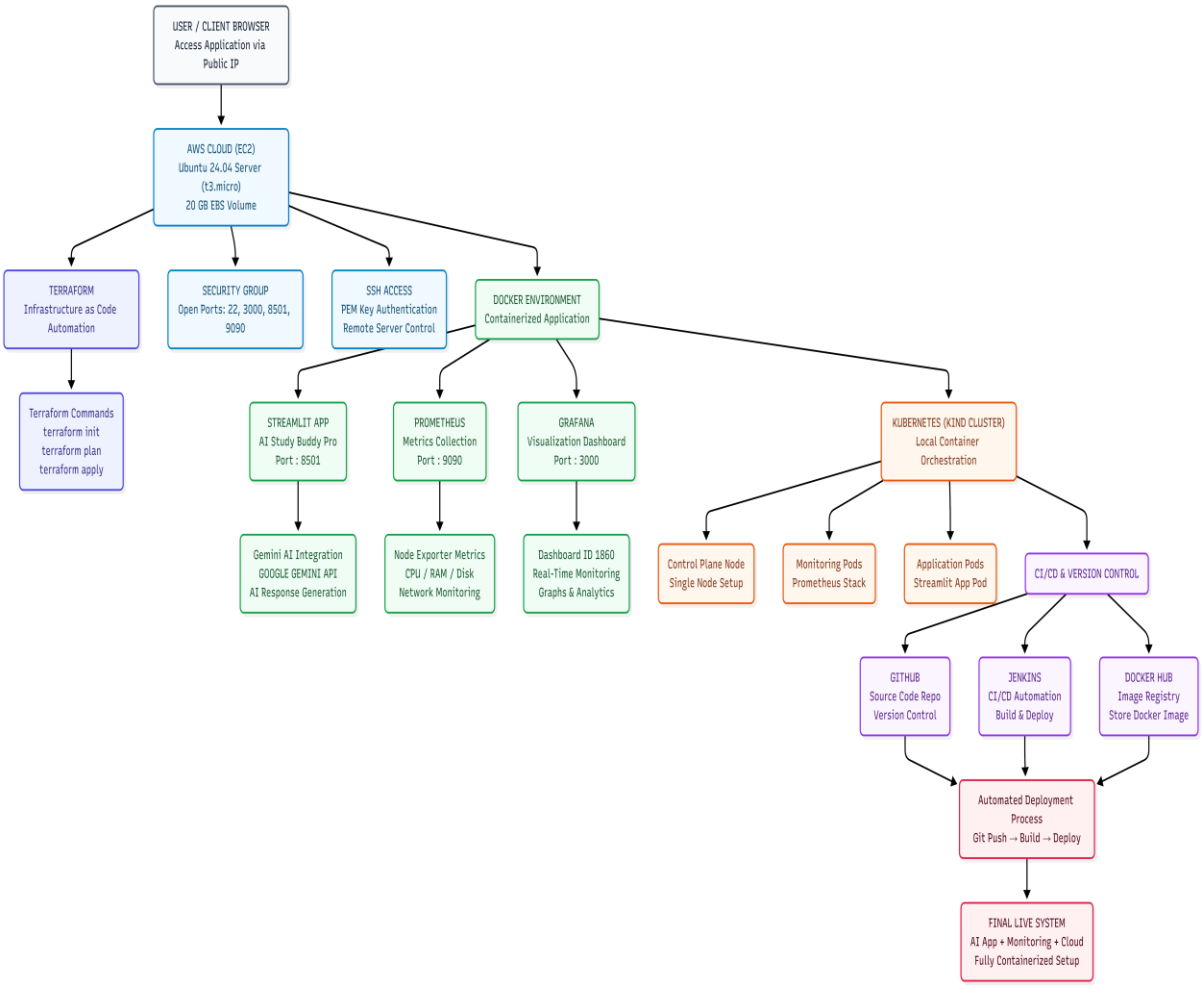
ABSTRACT

AI Study Buddy is a cloud-native intelligent educational platform developed using modern DevOps practices and cloud technologies. The project integrates Artificial Intelligence, containerization, orchestration, infrastructure automation, continuous integration, and monitoring into a single professional deployment workflow.

The application was developed using Python and Streamlit with Gemini API integration for AI-powered learning assistance. Docker was used for containerization while Kubernetes handled orchestration and scalability. Jenkins automated the CI/CD pipeline and Terraform automated AWS infrastructure provisioning. Monitoring and visualization were implemented using Prometheus and Grafana.

The project demonstrates a complete end-to-end DevOps lifecycle from local development to cloud deployment on AWS EC2. The system supports automated deployment, scalability, monitoring, infrastructure management, and production-style cloud hosting.

This project helped in understanding real-world DevOps workflows, cloud computing, infrastructure as code, monitoring systems, container orchestration, and deployment automation.



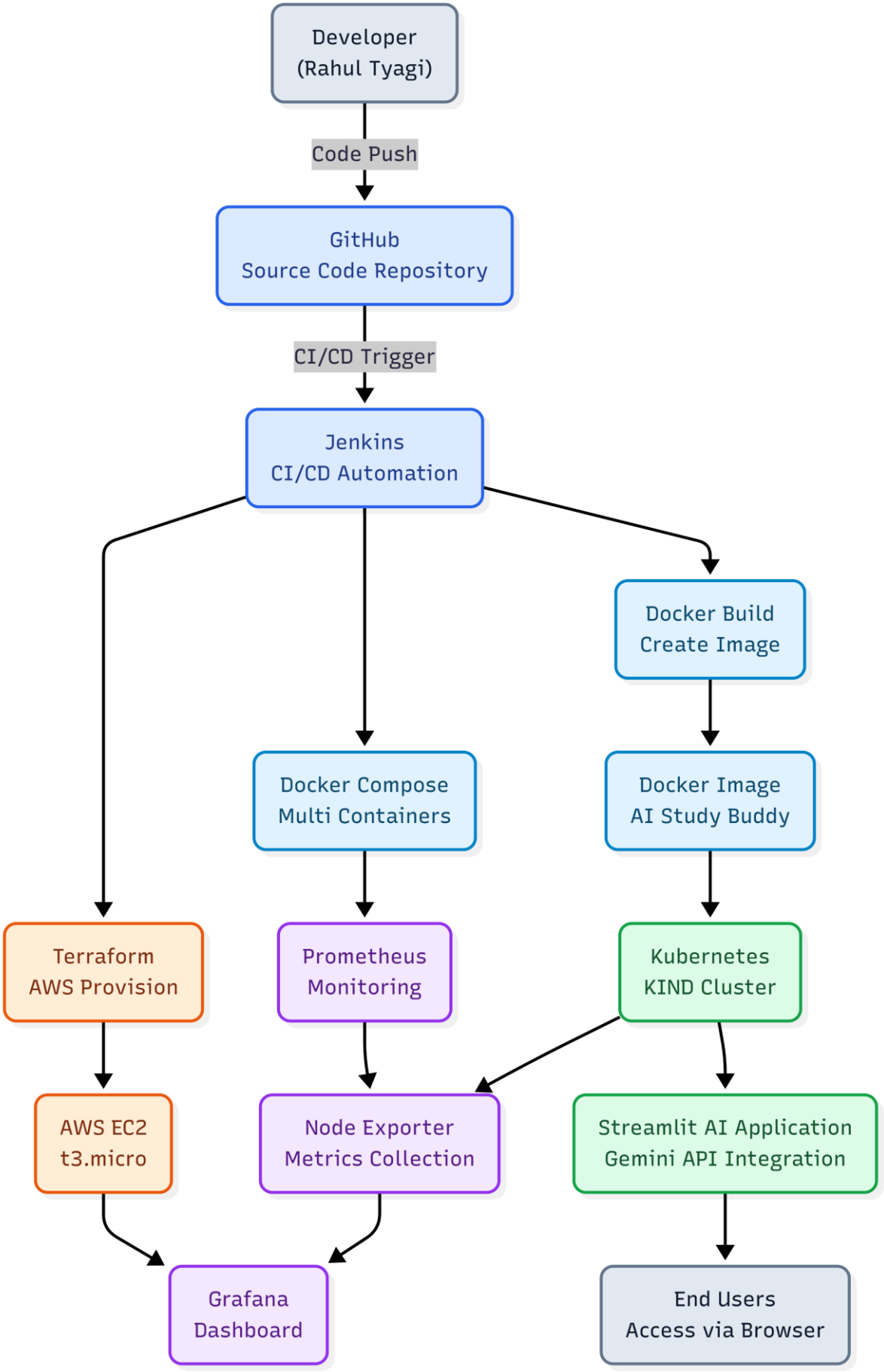

Developer
(Rahul Tyagi)
Code Push
GitHub
Source Code Repository
CI/CD Trigger
Jenkins
CI/CD Automation
Docker Build
Create Image
Docker Compose
Multi Containers
Docker Image
AI Study Buddy
Terraform
AWS Provision
Prometheus
Monitoring
Kubernetes
KIND Cluster
AWS EC2
t3.micro
Node Exporter
Metrics Collection
Streamlit AI Application
Gemini API Integration
Grafana
Dashboard
End Users
Access via Browser

TABLE OF CONTENTS

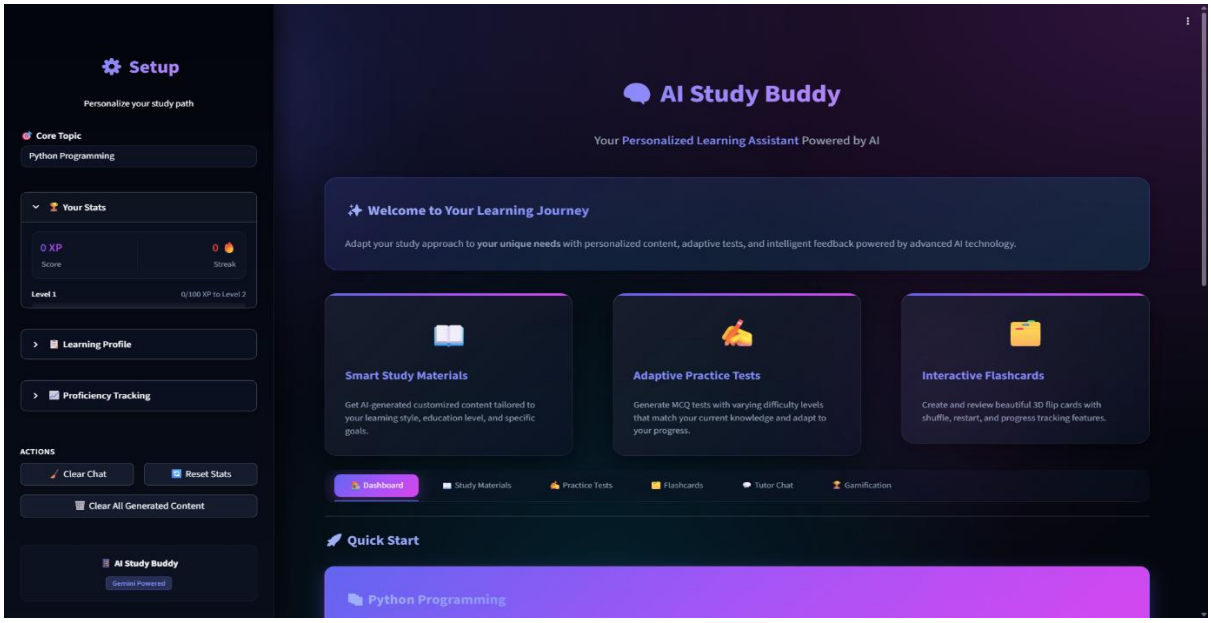
1. INTRODUCTION

Modern educational systems require intelligent, scalable, and cloud-based solutions capable of providing personalized assistance to students. Traditional learning systems often lack automation, scalability, intelligent assistance, and proper monitoring capabilities.

AI Study Buddy is designed to solve these issues by integrating Artificial Intelligence with modern DevOps and Cloud technologies. The platform allows students to interact with an AI-powered assistant capable of providing educational guidance and study assistance.

Key DevOps Practices Implemented

- Source code management
- CI/CD automation
- Containerization
- Orchestration
- Monitoring
- Cloud deployment
- Infrastructure automation



2. PROBLEM STATEMENT

Traditional educational applications face several limitations that hinder their effectiveness in modern deployment environments:

- Lack of scalability
- Manual deployment process
- No automated monitoring
- Difficulty in managing infrastructure

- Absence of CI/CD pipelines
- Poor cloud integration
- No intelligent learning support

Managing deployment manually becomes increasingly difficult as applications grow in complexity. Modern organisations require automated, scalable, and monitored systems. Therefore, there was a clear need to develop:

- An AI-powered educational platform
- A fully containerised architecture
- An automated deployment workflow
- A cloud-native infrastructure
- A real-time monitoring system

3. OBJECTIVES

Primary Objectives

- Develop an AI-powered learning platform
- Deploy application using Docker containers
- Implement Kubernetes orchestration
- Automate CI/CD using Jenkins
- Provision infrastructure using Terraform
- Deploy project on AWS cloud
- Implement monitoring using Prometheus and Grafana

Secondary Objectives

- Learn DevOps lifecycle
- Understand cloud-native architecture
- Implement Infrastructure as Code (IaC)
- Gain hands-on Kubernetes experience
- Learn production-style deployment practices

4. SCOPE OF PROJECT

Educational Assistance

- AI-powered learning guidance
- Interactive educational support

DevOps Implementation

- Docker containerisation
- Kubernetes orchestration
- Jenkins automation

- Monitoring setup

Cloud Deployment

- AWS EC2 deployment
- Terraform infrastructure automation

Monitoring and Analytics

- Real-time monitoring
- Dashboard visualisation
- Performance tracking

The project can be extended further with authentication systems, multi-user support, database integration, advanced analytics, load balancing, and auto-scaling.

5. EXISTING SYSTEM

Most traditional educational applications have significant shortcomings:

- Require manual deployment
- Are difficult to scale
- Lack cloud integration
- Do not support containerisation
- Lack automated monitoring

These systems are often difficult to maintain, infrastructure-dependent, and not production-ready.

6. PROPOSED SYSTEM

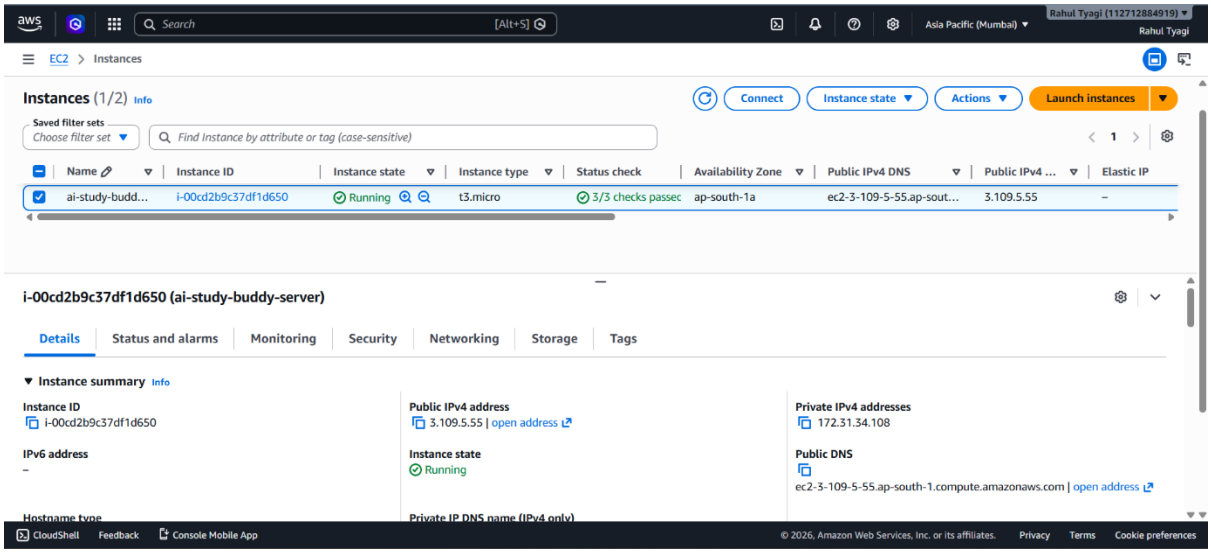
The proposed system introduces a modern cloud-native DevOps architecture with the following features:

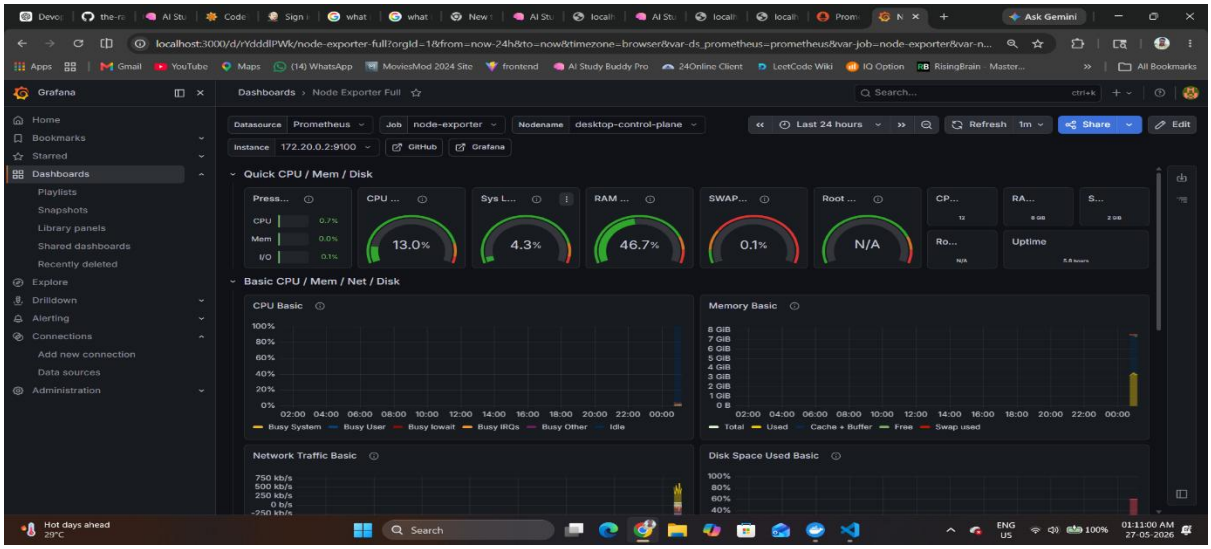
- AI-powered learning assistant
- Dockerised deployment
- Kubernetes orchestration
- Jenkins CI/CD automation
- Terraform infrastructure provisioning
- AWS cloud hosting
- Monitoring with Prometheus
- Visualisation using Grafana

System Workflow

```
User → Streamlit App → Gemini API
↓
Docker Container
↓
Kubernetes Cluster
↓
AWS EC2 Deployment
↓
Prometheus + Grafana Monitoring
```

```
(.venv) PS C:\Users\rahul\Downloads\AI_Study_Buddy_Pro> kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS      AGE
alertmanager-kube-prometheus-stack-alertmanager-0        2/2     Running   2 (67s ago)   12h
kube-prometheus-stack-grafana-79bc64c748-q45gc           3/3     Running   3 (67s ago)   12h
kube-prometheus-stack-kube-state-metrics-6b747b7c5f-zlbdh 1/1    Running   2 (15s ago)   12h
kube-prometheus-stack-operator-7c9b5bb96c-pz8s5          1/1     Running   2 (21s ago)   12h
kube-prometheus-stack-prometheus-node-exporter-rnh5n     1/1     Running   1 (67s ago)   12h
prometheus-kube-prometheus-stack-prometheus-0            2/2     Running   2 (67s ago)   12h
(.venv) PS C:\Users\rahul\Downloads\AI_Study_Buddy_Pro>
```





7. TECHNOLOGY STACK

The project uses multiple modern technologies across frontend development, backend, DevOps, cloud computing, monitoring, and infrastructure automation.

7.1 Frontend Technology — Streamlit

Streamlit was used to develop the user interface of the AI Study Buddy application.

Why Streamlit?

- Easy integration with Python
- Lightweight framework
- Rapid prototyping
- Suitable for AI applications

7.2 Backend Technology — Python

Python was used as the core backend language.

Libraries Used

- streamlit
- google-generativeai
- requests
- dotenv

7.3 AI Integration — Gemini API

Google Gemini API was integrated to provide intelligent educational assistance.

Functionality

- Question answering
- Study support
- AI-powered responses

Benefits

- Fast response generation
- Natural language interaction
- Educational guidance

8. VERSION CONTROL USING GIT & GITHUB

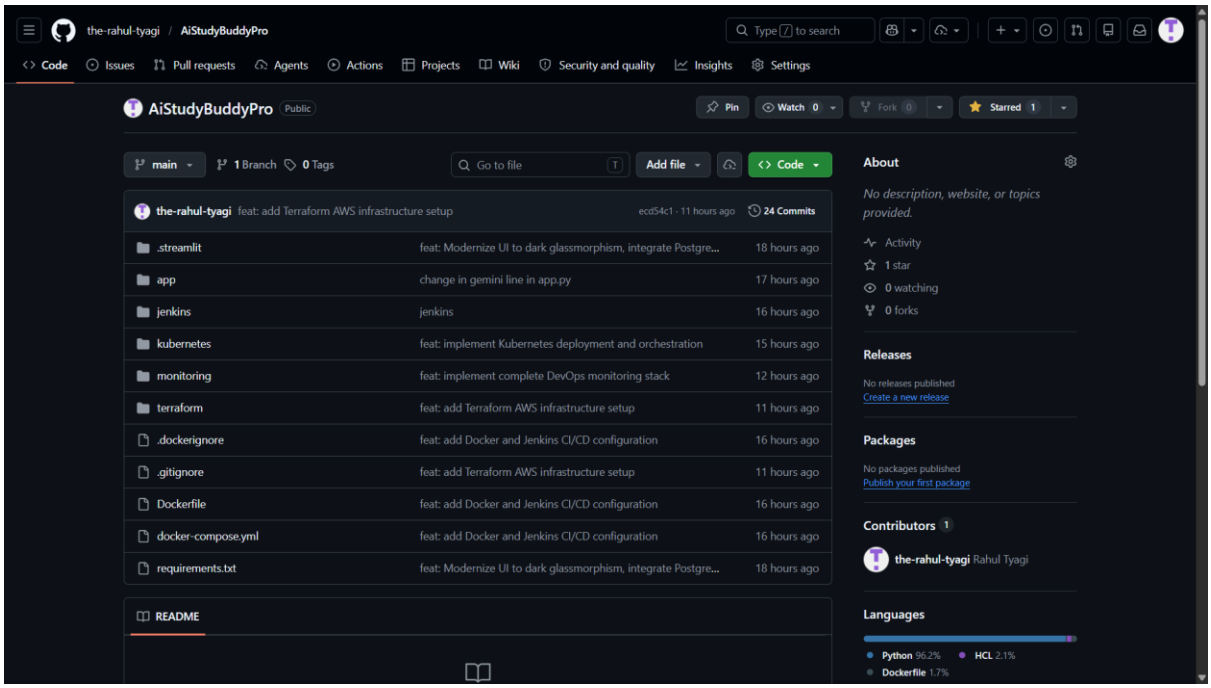
Git was used for version control while GitHub was used as the remote repository hosting platform.

Git Features Used

- Source code management — code tracking, version history, collaboration
- Branching — feature development, safe code management
- Remote repository — cloud backup, repository sharing

Common Git Commands Used

```
git init
git add .
git commit -m "Initial Commit"
git push origin main
git pull origin main
git clone <repo-url>
```



9. DOCKER CONTAINERISATION

Docker was used to containerise the application, ensuring consistency, portability, easy deployment, and dependency isolation.

Docker Architecture in Project

```
Application Code
↓
Docker Image
↓
Docker Container
↓
AWS EC2 Deployment
```

Dockerfile

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 8501
```

```
CMD ["streamlit", "run", "app/app.py"]
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

(.venv) PS C:\Users\rahul\Downloads\AI_Study_Buddy_Pro> docker compose build
#12 exporting to image
#12 exporting layers done
#12 exporting manifest sha256:bf58b34befa65effe78c06cd566babd500d1d54c6ec6f96669864eef323cb616 done
#12 exporting config sha256:65f21371321232c6e5be3e95cc41f88a4c3a260a436bfea9e1ffadc95508323c done
#12 exporting attestation manifest sha256:ac8d65a187e6cbb3c2452f8ca6195de0f4a1c44193cacd44c4fc00712677aeb8
#12 exporting attestation manifest sha256:ac8d65a187e6cbb3c2452f8ca6195de0f4a1c44193cacd44c4fc00712677aeb8 0.0s done
#12 exporting manifest list sha256:708fadc40503a39077ad6e7b1b2d7c23053cc7f0ab57fb65e797bc209039a15a 0.0s done
#12 naming to docker.io/rahultyagi/ai-study-buddy-pro:latest done
#12 unpacking to docker.io/rahultyagi/ai-study-buddy-pro:latest 0.0s done
#12 DONE 0.2s

#13 resolving provenance for metadata file
#13 DONE 0.1s
[+] build 1/1
 ✓ Image rahultyagi/ai-study-buddy-pro:latest Built
(.venv) PS C:\Users\rahul\Downloads\AI_Study_Buddy_Pro>
```

Docker Commands Reference

```
docker build -t ai-study-buddy .        # Build image
docker run -p 8501:8501 ai-study-buddy # Run container
docker ps                               # List running containers
docker stop <container-id>              # Stop container
docker rm <container-id>                # Remove container
```

```
ubuntu@ip-172-31-34-108:~/AiStudyBuddyPro$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED       STATUS                    PORTS
                    NAMES
c19dac1f6930   rahultyagi/ai-study-buddy-pro:latest   "streamlit run app/a…"   4 hours ago   Up About an hour (healthy)   0.0.0.0:8501->8501/tc
p, [::]:8501->8501/tcp   ai-study-buddy-container
ubuntu@ip-172-31-34-108:~/AiStudyBuddyPro$
```

10. DOCKER COMPOSE

Docker Compose was used for multi-container deployment, managing the AI application, Prometheus, and Grafana simultaneously.

```
docker compose up -d    # Start all services
docker compose down     # Stop all services
docker compose logs     # View logs
```

```
ubuntu@ip-172-31-34-108:~/AiStudyBuddyPro$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                     STATUS                    PORTS
ai-study-buddy-container   Up About an hour (healthy)   0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
ubuntu@ip-172-31-34-108:~/AiStudyBuddyPro$
```

11. KUBERNETES ORCHESTRATION

Kubernetes was used for container orchestration and scaling. It provides auto-restart, scaling, load balancing, and self-healing capabilities.

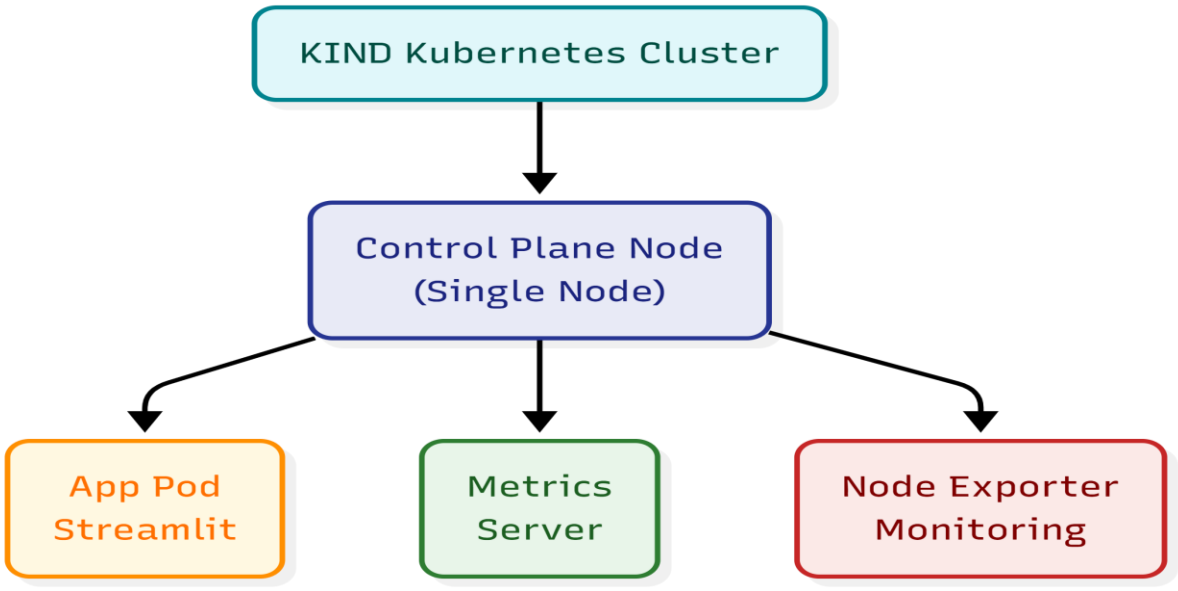
Kubernetes Components Used

- Deployment — manages application replicas
- Service — exposes application to users
- Pods — runs containers
- Metrics Server — collects cluster metrics

Kubernetes Commands Reference

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl get svc
kubectl scale deployment ai-study-buddy --replicas=2
kubectl delete -f deployment.yaml
kubectl top nodes
kubectl top pods
```

```
(.venv) PS C:\Users\rahul\Downloads\AI_Study_Buddy_Pro> kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS      AGE
alertmanager-kube-prometheus-stack-alertmanager-0        2/2     Running   2 (67s ago)   12h
kube-prometheus-stack-grafana-79bc64c748-q45gc           3/3     Running   3 (67s ago)   12h
kube-prometheus-stack-kube-state-metrics-6b747b7c5f-zlbdh 1/1    Running   2 (15s ago)   12h
kube-prometheus-stack-operator-7c9b5bb96c-pz8s5          1/1     Running   2 (21s ago)   12h
kube-prometheus-stack-prometheus-node-exporter-rnh5n     1/1     Running   1 (67s ago)   12h
prometheus-kube-prometheus-stack-prometheus-0            2/2     Running   2 (67s ago)   12h
(.venv) PS C:\Users\rahul\Downloads\AI_Study_Buddy_Pro>
```



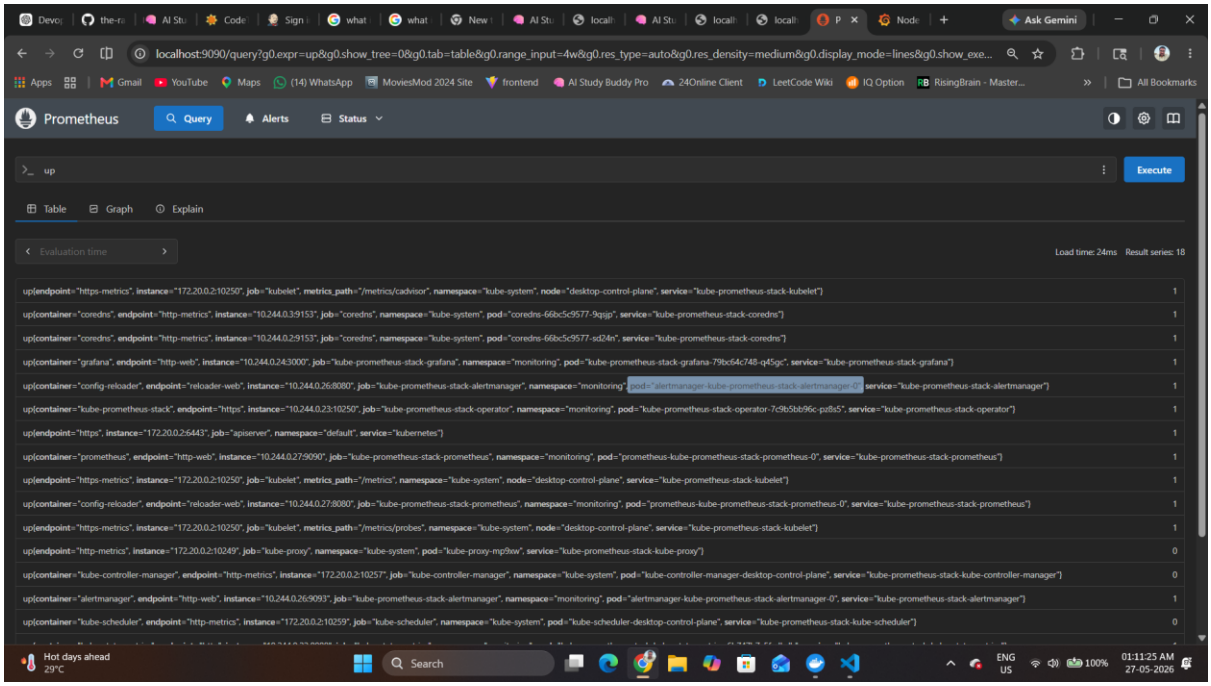
12. PROMETHEUS MONITORING

Prometheus was used for metrics collection, monitoring, and performance analysis — covering CPU, memory, pod, and node monitoring.

Node Exporter Installation via Helm

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install node-exporter prometheus-community/prometheus-node-exporter -n monitoring
```

Prometheus UI is accessible at: http://<EC2-IP>:9090

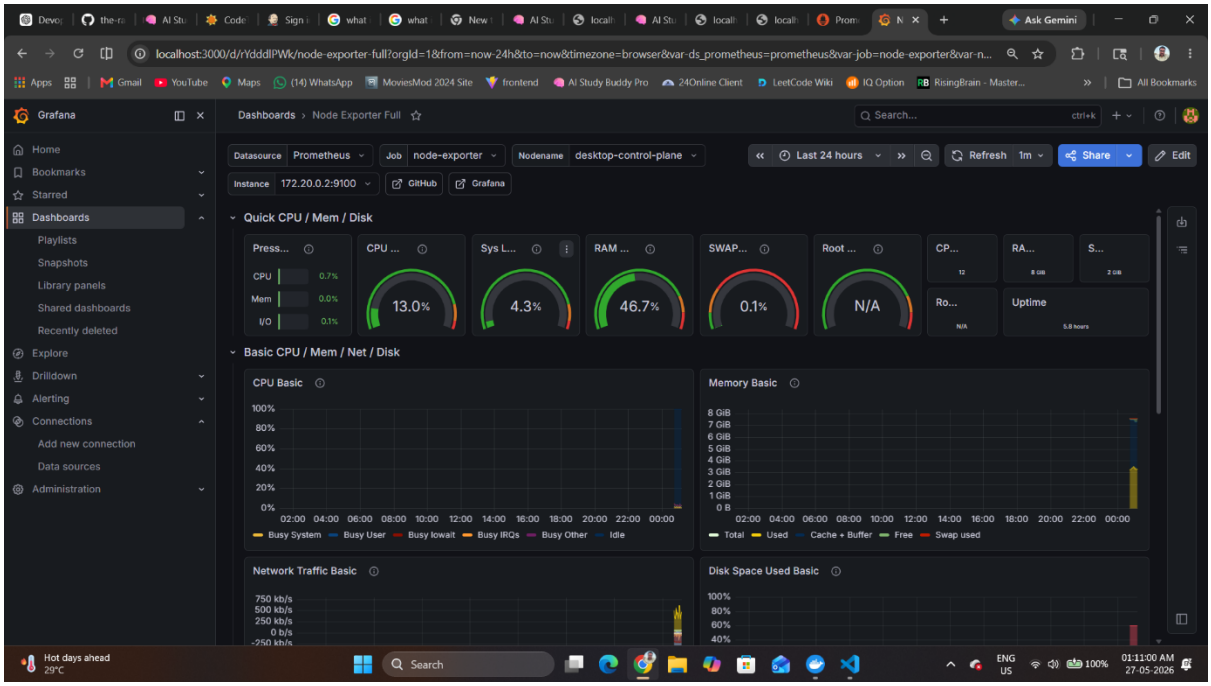


13. GRAFANA DASHBOARD

Grafana was used for visualisation, dashboards, and monitoring analytics.

Parameter	Value
Dashboard ID	1860
Dashboard Name	Node Exporter Full
Grafana Port	3000
Default Username	admin
Default Password	admin

```
Node Exporter → Prometheus → Grafana Dashboard
```



14. TERRAFORM INFRASTRUCTURE AUTOMATION

Terraform was used for Infrastructure as Code (IaC), automating EC2 creation, security group creation, and infrastructure provisioning.

Terraform Files

- main.tf — defines AWS infrastructure
- variables.tf — stores variables
- outputs.tf — displays outputs

```
terraform init      # Initialise
terraform plan      # Generate plan
terraform apply     # Apply infrastructure
terraform destroy   # Destroy infrastructure
```

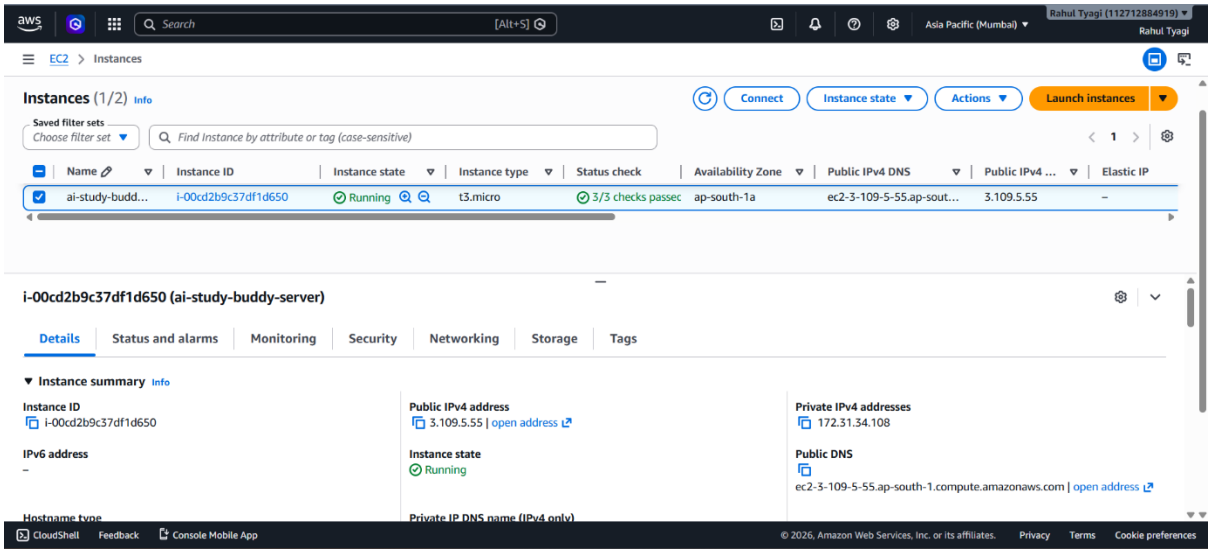
15. AWS CLOUD DEPLOYMENT

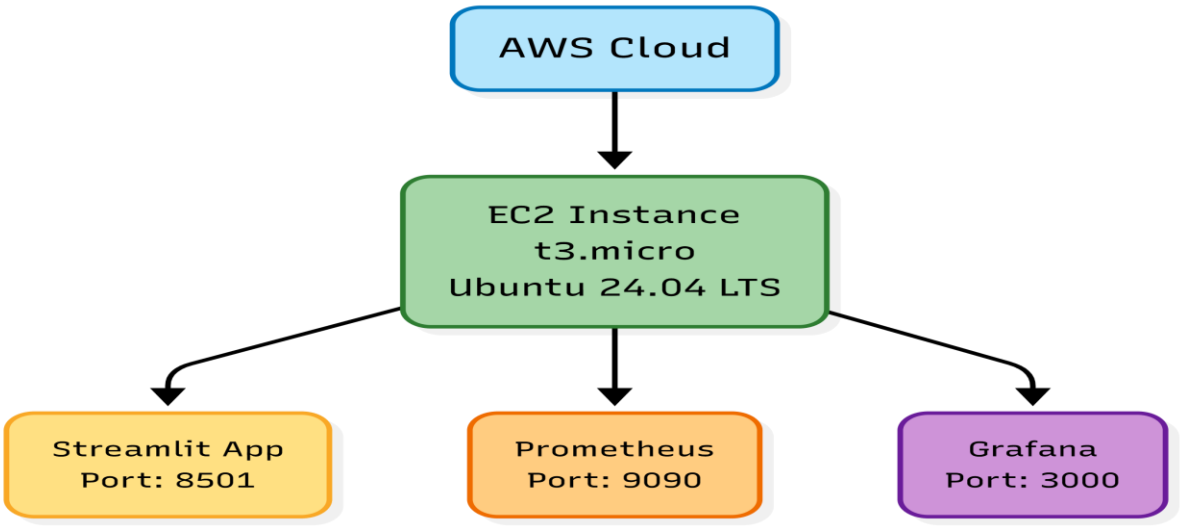
Amazon Web Services (AWS) was used for cloud deployment.

Service	Purpose
EC2	Virtual machine hosting
Security Groups	Firewall configuration
EBS Volume	Storage management

Instance Type	t3.micro
Region	ap-south-1
Operating System	Ubuntu 24.04

```
aws configure                # Configure credentials
aws sts get-caller-identity  # Verify account
```





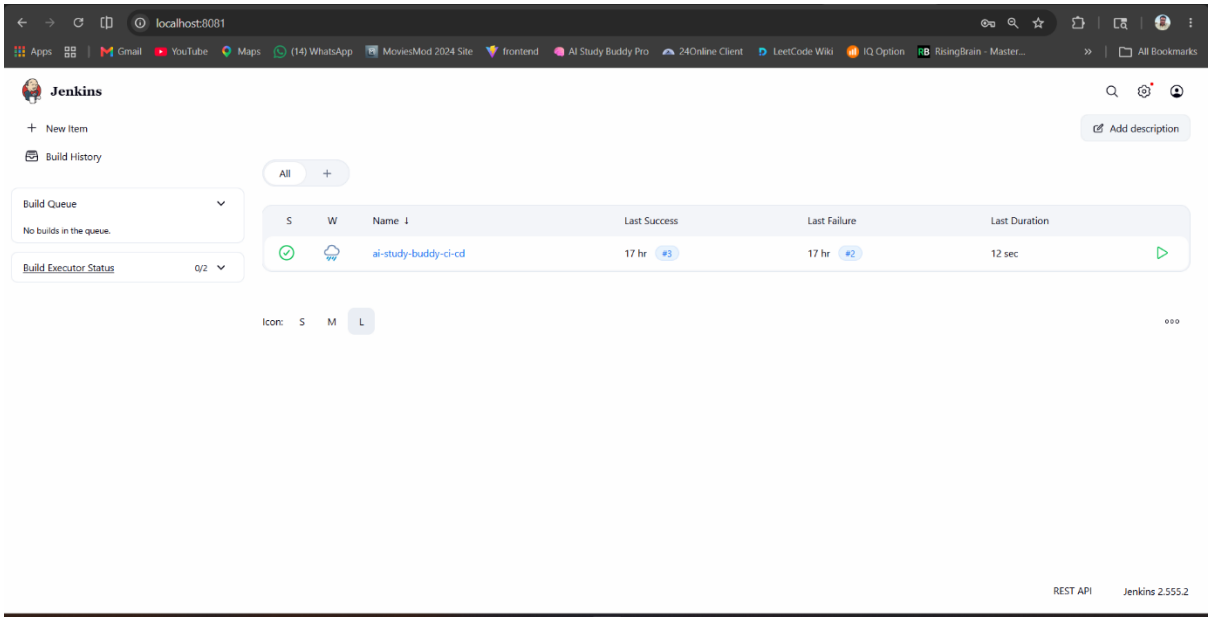
16. CI/CD PIPELINE USING JENKINS

Jenkins was used to automate the build process, testing, and deployment.

```
Developer Pushes Code  →  GitHub Repository  →  Jenkins Pipeline Triggered
→  Docker Image Built  →  Container Updated  →  Application Redeployed
```

Pipeline Stages

1. Clone Repository
2. Build Docker Image
3. Run Container
4. Deploy Application



17. PROJECT IMPLEMENTATION PROCESS

The project was developed in multiple phases from application development to complete cloud deployment.

17.1 Initial Project Setup

- Created project folder structure
- Initialised Git repository
- Created Python virtual environment
- Installed dependencies

```
python -m venv .venv
.venv\Scripts\activate    # Windows PowerShell
pip install -r requirements.txt
```

17.2 Application Development

- AI Chat Interface — users interact with the AI assistant
- Gemini API Integration — AI generates educational responses
- Streamlit Frontend — provides interactive UI components

Project Folder Structure

```
AI_Study_Buddy_Pro/
│
├── app/
│   └── app.py
│
├── kubernetes/
├── monitoring/
├── terraform/
├── jenkins/
│
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── .env
```

18. DOCKER IMPLEMENTATION PROCESS

5. Create Dockerfile
6. Build Docker image: docker build -t ai-study-buddy .
7. Run container: docker run -p 8501:8501 ai-study-buddy
8. Verify: docker ps

```
ubuntu@ip-172-31-34-108:~/AiStudyBuddyPro$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES                    STATUS                    PORTS
ai-study-buddy-container   Up About an hour (healthy)   0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
ubuntu@ip-172-31-34-108:~/AiStudyBuddyPro$
```

19. DOCKER COMPOSE IMPLEMENTATION

```
services:
app:
build: .
ports: ["8501:8501"]
prometheus:
image: prom/prometheus
grafana:
image: grafana/grafana

docker compose up -d
```

20. KUBERNETES IMPLEMENTATION

20.1 Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: ai-study-buddy
spec:
replicas: 2
...
```

20.2 Service YAML

```
apiVersion: v1
kind: Service
metadata:
name: ai-study-buddy-service
...
```

20.3 Deploy & Verify

```
kubectl apply -f deployment.yaml
kubectl get pods
kubectl get svc
```

21. METRICS SERVER IMPLEMENTATION

Problem

Metrics Server pods remained at 0/1 Running. kubectl top nodes was not working.

Root Cause

TLS verification issue between metrics-server and kubelet.

Solution

Added --kubelet-insecure-tls inside metrics-server deployment args.

```
kubectl edit deployment metrics-server -n kube-system
kubectl rollout restart deployment metrics-server -n kube-system
kubectl delete pod -n kube-system -l k8s-app=metrics-server

kubectl top nodes     # Verify
kubectl top pods -A
```

22. PROMETHEUS IMPLEMENTATION

```
kubectl create namespace monitoring
kubectl apply -f prometheus.yaml
kubectl get pods -n monitoring
```

23. GRAFANA IMPLEMENTATION

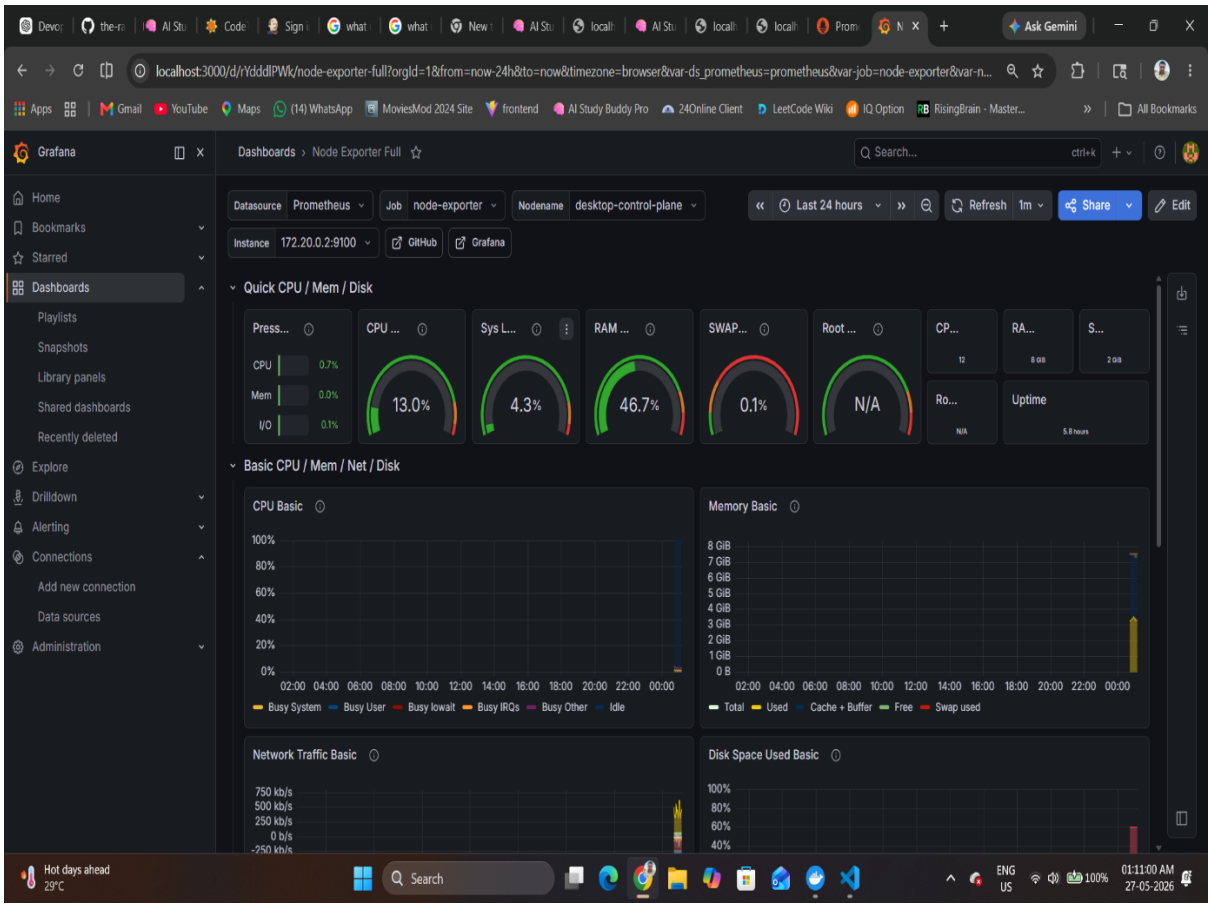
Access Grafana at http://localhost:3000 using default credentials: admin / admin

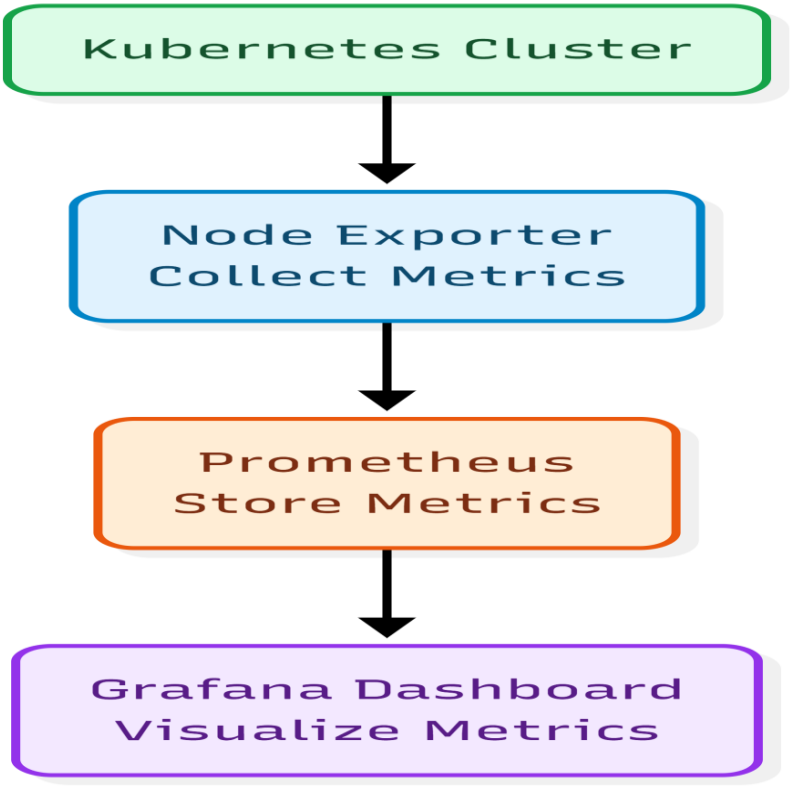
23.1 Add Prometheus Data Source

- Open Grafana → Connections → Data Sources
- Add Prometheus
- Prometheus URL: http://prometheus-service:9090

23.2 Import Dashboard

Import Dashboard ID 1860 (Node Exporter Full) for CPU, RAM, Disk, and Network monitoring.





24. HELM IMPLEMENTATION

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install node-exporter prometheus-community/prometheus-node-exporter -n monitoring
kubectl get pods -n monitoring
```

25. TERRAFORM IMPLEMENTATION

25.1 Configure AWS CLI

```
aws configure
aws sts get-caller-identity
```

25.2 Initialisation

Initial error: "No configuration files". Solution: Created main.tf, variables.tf, and outputs.tf.

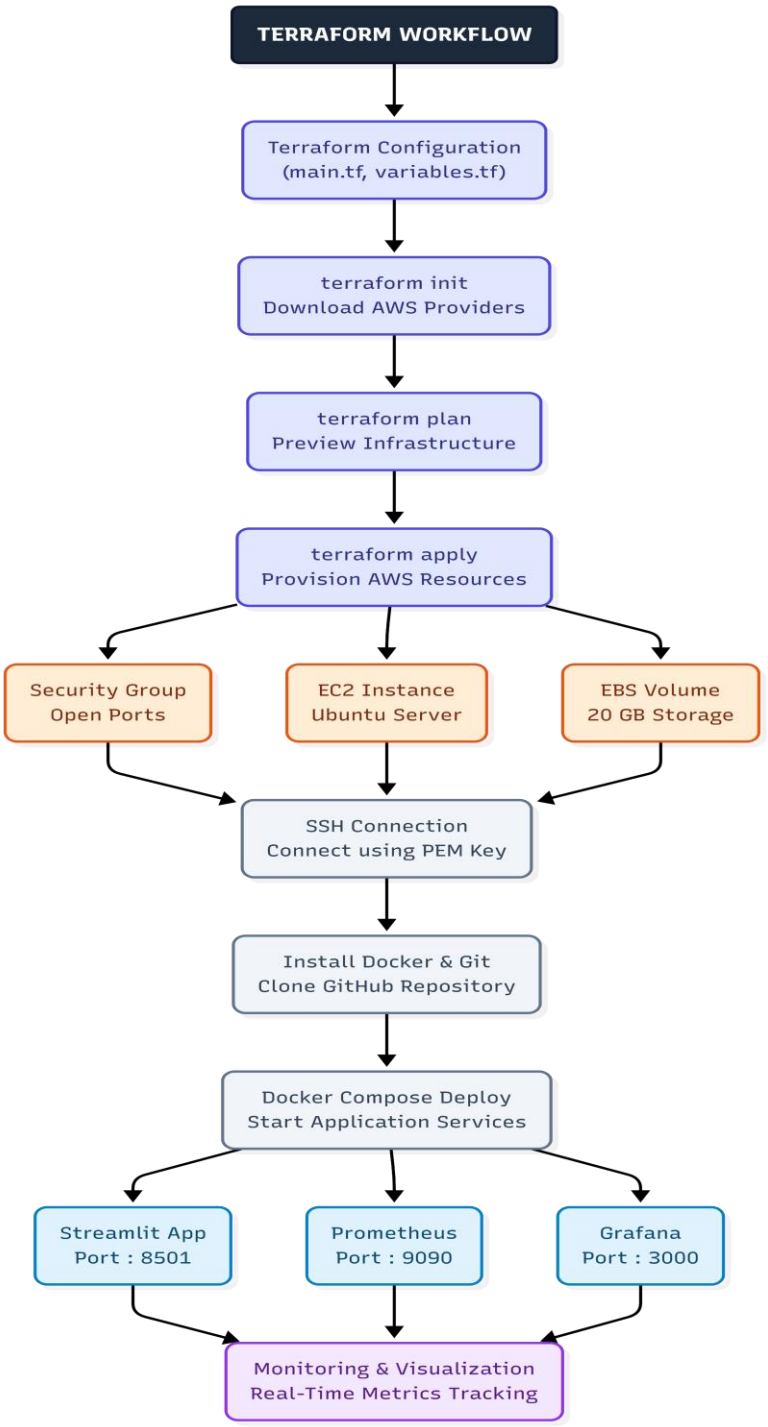
```
terraform init
```

25.3 Apply Infrastructure

```
terraform apply
```

Infrastructure Created

- EC2 instance
- Security group
- Storage volume



26. AWS EC2 DEPLOYMENT PROCESS

```
ssh -i "ai-study-buddy-key.pem" ubuntu@<EC2-IP>
```

26.1 Docker Permission Issue

Error: permission denied while trying to connect to docker.sock

```
sudo usermod -aG docker ubuntu
newgrp docker
docker ps
```

27. APPLICATION DEPLOYMENT ON AWS

```
git clone https://github.com/the-rahul-tyagi/AiStudyBuddyPro.git
cd AiStudyBuddyPro
nano .env
GEMINI_API_KEY=your_api_key
docker compose up -d
```

Application accessible at: http://<EC2-IP>:8501

28. STORAGE EXPANSION ISSUE

Problem

Error: no space left on device. Prometheus and Grafana images exceeded the default 8 GB EBS volume.

Solution — Expand EBS Volume from 8 GB to 20 GB

```
sudo growpart /dev/nvme0n1 1
sudo resize2fs /dev/nvme0n1p1
df -h
```

29. FINAL APPLICATION URLS

Service	URL
AI Study Buddy	http://<EC2-IP>:8501
Grafana Dashboard	http://<EC2-IP>:3000
Prometheus Dashboard	http://<EC2-IP>:9090

30. CHALLENGES FACED DURING PROJECT DEVELOPMENT

During the development and deployment of AI Study Buddy, multiple technical challenges were encountered. Solving these issues provided hands-on industry-level DevOps experience. This section covers each problem, its root cause, troubleshooting steps, and the final solution applied.

30.1 Metrics Server Not Working in Kubernetes

Symptoms

```
metrics-server-xxxx   0/1 Running
kubectl top nodes  →  Error
```

Root Cause

Metrics Server could not communicate securely with kubelet due to TLS certificate verification problems.

Solution

9. Check deployment: kubectl get deployment metrics-server -n kube-system -o yaml
10. Edit deployment: kubectl edit deployment metrics-server -n kube-system
11. Add --kubelet-insecure-tls to container args
12. Restart: kubectl rollout restart deployment metrics-server -n kube-system

30.2 Grafana Dashboard Not Showing Metrics

Symptoms

- "No Data" message
- Dropdowns empty
- Node exporter missing

Root Cause

Prometheus data source and Node Exporter metrics were not configured correctly.

Resolution Steps

13. Install Helm: winget install Helm.Helm
14. Restart PowerShell to refresh PATH
15. Add Helm repository and install Node Exporter
16. Configure Grafana data source: http://prometheus-service:9090
17. Import Dashboard ID 1860

30.3 Helm Command Not Recognised

Error

```
helm : The term 'helm' is not recognized
```

Root Cause & Solution

Environment PATH was not refreshed after installation. Solution: Close and reopen PowerShell terminal.

30.4 Terraform Configuration Error

Error

```
No configuration files
```

Solution

Created main.tf, variables.tf, and outputs.tf, then ran terraform init → validate → plan → apply.

30.5 SSH Connection Timeout Issue

Error

```
ssh: connect to host port 22: Connection timed out
```

Root Cause & Solution

Old EC2 instance IP was used after infrastructure was recreated by Terraform. Used the updated public IP from terraform apply output.

30.6 Docker Permission Denied Issue

Error

```
permission denied while trying to connect to docker.sock
```

Root Cause & Solution

Ubuntu user was not part of the docker group.

```
sudo usermod -aG docker ubuntu
newgrp docker
docker ps
```

30.7 Missing GEMINI_API_KEY Error

Error

```
GEMINI_API_KEY not found in .env file
```

Solution

```
nano .env
GEMINI_API_KEY=your_api_key
```

30.8 Docker Compose Flag Error

Error

```
unknown shorthand flag: 'd' in -d
```

Solution

Used "docker compose up -d" (with space) instead of "docker-compose up -d" (with hyphen).

30.9 Grafana & Prometheus Not Opening on AWS

Problem

Only the Streamlit app (port 8501) was accessible. Grafana (3000) and Prometheus (9090) were unreachable.

Root Cause & Solution

Containers were not running and ports were not mapped in docker-compose.yml. Updated the file with proper service definitions and port mappings.

30.10 Disk Storage Full Issue

Error

```
no space left on device
```

Root Cause & Solution

Prometheus and Grafana Docker images exceeded the default 8 GB EBS volume. Expanded to 20 GB:

18. Modify EBS Volume in AWS Console from 8 GB to 20 GB
19. sudo growpart /dev/nvme0n1 1
20. sudo resize2fs /dev/nvme0n1p1
21. Verify with df -h

31. MAJOR LEARNINGS FROM TROUBLESHOOTING

Kubernetes Debugging

- Pod troubleshooting
- Deployment editing
- Metrics server debugging

Docker Management

- Container lifecycle
- Image handling
- Docker permissions
- Compose management

AWS Infrastructure

- EC2 deployment
- SSH access
- Storage expansion
- Security groups

Monitoring Stack

- Prometheus metrics
- Grafana dashboards
- Node Exporter

Terraform Automation

- Infrastructure as code
- Cloud provisioning
- Automation workflows

32. OVERALL TROUBLESHOOTING IMPACT

The troubleshooting phase significantly improved practical understanding of production-level deployment environments. The project demonstrated hands-on implementation of DevOps lifecycle, infrastructure automation, monitoring systems, cloud deployment, and real-world debugging techniques used in industry.

33. CI/CD AUTOMATION USING JENKINS

33.1 Introduction to CI/CD

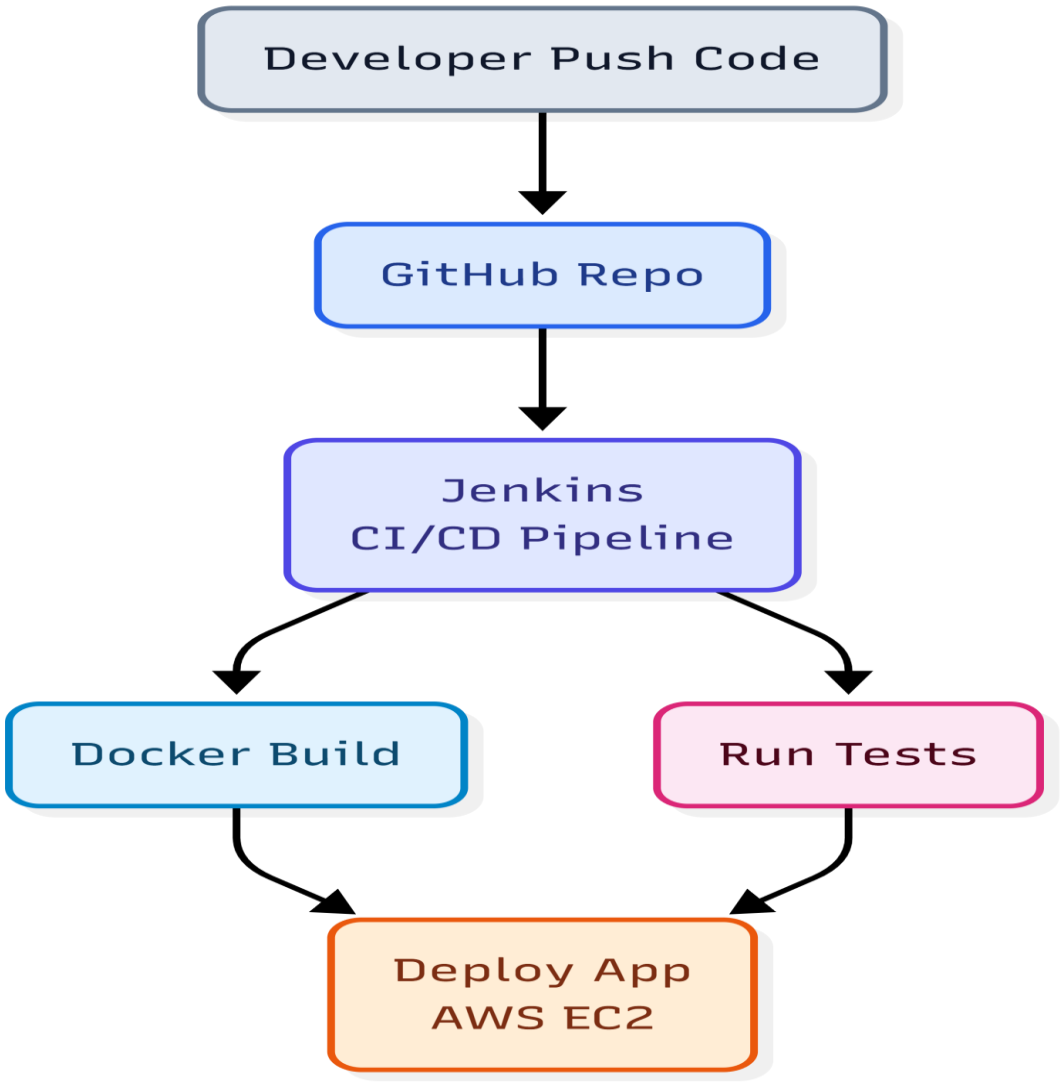
CI/CD stands for Continuous Integration and Continuous Deployment/Delivery. It automates code building, testing, Docker image creation, and deployment — reducing manual effort and improving delivery speed.

33.2 Why Jenkins Was Used

- Open-source and highly extensible
- Industry standard tool
- Easy integration with GitHub, Docker, and Kubernetes

33.3 CI/CD Workflow

```
Developer → GitHub Push → Jenkins Trigger → Build Docker Image
→ Push Image → Deploy Application → Monitoring
```



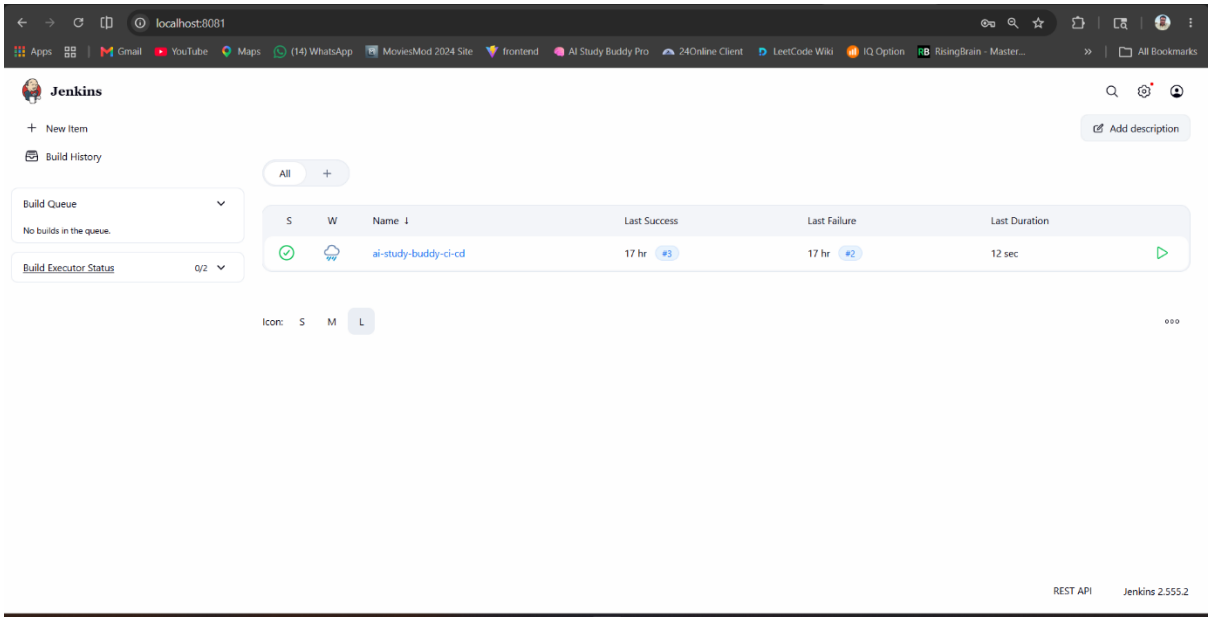
33.4 Jenkins Setup Process

```
docker run -d \
--name jenkins \
-p 8080:8080 \
-p 50000:50000 \
jenkins/jenkins:lts

docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Plugins Installed

- Git Plugin
- Docker Plugin
- Pipeline Plugin
- Kubernetes Plugin



33.5 Jenkins Pipeline (Jenkinsfile)

```
pipeline {
agent any
stages {
stage('Clone Repository') {
steps { git 'https://github.com/the-rahul-tyagi/AiStudyBuddyPro.git' }
}
stage('Build Docker Image') {
steps { sh 'docker build -t ai-study-buddy .' }
}
stage('Run Container') {
steps { sh 'docker run -d -p 8501:8501 ai-study-buddy' }
}
}
}
```

33.6 Benefits Achieved Using Jenkins

- Automatic builds on code push
- Reduced manual deployment effort
- Faster delivery and easier debugging
- Repeatable, version-controlled deployments

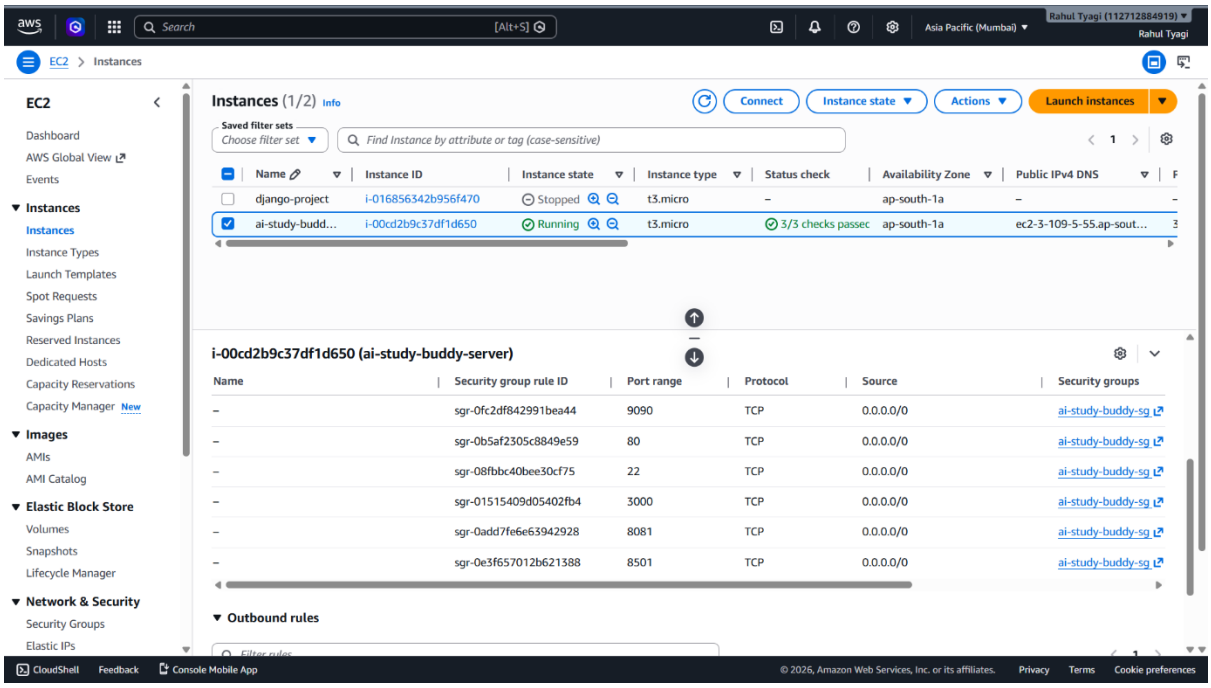
34. SECURITY IMPLEMENTATION

Several security measures were implemented across the project.

34.1 AWS Security Groups

Port	Purpose
22	SSH

Port	Purpose
8501	Streamlit App
3000	Grafana
9090	Prometheus



34.2 SSH Key-Based Authentication

SSH key authentication was used instead of passwords, providing encrypted authentication and protection against brute force attacks.

```
ssh -i "ai-study-buddy-key.pem" ubuntu@<EC2-IP>
```

34.3 Environment Variables Security

Sensitive API keys were not hardcoded. Stored inside .env file:

```
GEMINI_API_KEY=xxxxxxxxxxxx
```

34.4 Docker Isolation

Docker containers isolate application runtime environments, providing secure dependency management and reduced host conflicts.

34.5 Infrastructure as Code Security

Terraform provides reproducible, version-controlled cloud setup ensuring consistent and auditable deployments.

35. AUTOMATION STATUS OF PROJECT

Automated Features

- Docker containerisation
- Kubernetes deployment
- Monitoring setup
- Terraform infrastructure provisioning
- Jenkins CI/CD pipeline
- Cloud deployment

Manual Steps Still Required

- Push code to GitHub after changes
- Rebuild Docker image
- Redeploy container manually

Full Automation Goal (Future)

- GitHub Webhooks for automatic Jenkins trigger
- Auto Docker build and push
- Automatic deployment after every push

36. FUTURE SCOPE OF PROJECT

36.1 Kubernetes Production Deployment

- Amazon EKS deployment
- Multi-node cluster
- Autoscaling

36.2 Domain & HTTPS

- Custom domain
- SSL certificates
- HTTPS encryption

36.3 Advanced CI/CD

- GitHub Actions
- Automated testing
- Rollback mechanism
- Blue-green deployment

36.4 Monitoring Enhancements

- AlertManager
- Email alerts and Slack notifications
- Centralised logging

36.5 AI Enhancements

- Personalised recommendations
- Voice interaction
- Multilingual support
- AI-generated quizzes

37. TECHNOLOGIES USED

37.1 Frontend

Technology	Purpose
Streamlit	Web Interface
HTML/CSS	UI Components

37.2 Backend

Technology	Purpose
Python	Backend Logic
Gemini API	AI Functionality

37.3 DevOps

Tool	Purpose
Git	Version Control
GitHub	Code Hosting
Docker	Containerisation
Docker Compose	Multi-container Management
Kubernetes	Container Orchestration
Jenkins	CI/CD
Terraform	Infrastructure Automation
Prometheus	Monitoring
Grafana	Visualisation

37.4 Cloud

Service	Purpose
AWS EC2	Hosting
AWS Security Groups	Firewall
AWS EBS	Storage

38. COMMANDS USED IN PROJECT

Git Commands

```
git init
git add .
git commit -m "Initial Commit"
git push origin main
```

Docker Commands

```
docker build -t ai-study-buddy .
docker run -p 8501:8501 ai-study-buddy
docker ps
docker images
```

Kubernetes Commands

```
kubectl get pods
kubectl apply -f deployment.yaml
kubectl get services
kubectl top pods
```

Terraform Commands

```
terraform init
terraform validate
terraform plan
terraform apply
terraform destroy
```

AWS Commands

```
aws configure
aws sts get-caller-identity
```

Monitoring Commands

```
helm repo add prometheus-community ...
helm install node-exporter ...
kubectl get pods -n monitoring
```

39. PROJECT OUTCOMES

Technical Outcomes

- AI-powered study assistant created

- Dockerised application
- Kubernetes deployment implemented
- Monitoring stack integrated
- AWS cloud deployment completed
- Terraform infrastructure automation completed
- Jenkins CI/CD pipeline implemented

Learning Outcomes

- Cloud Computing
- DevOps
- Monitoring
- Containerisation
- Infrastructure Automation
- CI/CD Pipelines
- Kubernetes

40. CONCLUSION

The AI Study Buddy project successfully demonstrated the integration of Artificial Intelligence with modern DevOps and Cloud technologies.

The project involved application development, Docker containerisation, Kubernetes orchestration, monitoring implementation, infrastructure automation, cloud deployment, and CI/CD automation. Multiple real-world deployment challenges were solved during development, providing practical industry-level experience.

The Final System Is

- Scalable
- Portable
- Monitored
- Cloud-ready
- Automation-focused

This project significantly enhanced practical knowledge of DevOps Engineering, Cloud Infrastructure, Monitoring Systems, Automation Workflows, and Production Deployment Practices. It serves as a strong foundation for understanding how AI applications integrate with professional DevOps workflows.

41. REFERENCES

Technology	Official Documentation
Docker	https://docs.docker.com
Kubernetes	https://kubernetes.io/docs
Terraform	https://developer.hashicorp.com/terraform/docs
Jenkins	https://www.jenkins.io/doc/
AWS	https://docs.aws.amazon.com
Prometheus	https://prometheus.io/docs/
Grafana	https://grafana.com/docs/
Streamlit	https://docs.streamlit.io

42. APPENDIX

Important URLs

Service	URL
Streamlit App	http://<EC2-IP>:8501
Grafana	http://<EC2-IP>:3000
Prometheus	http://<EC2-IP>:9090
Jenkins	http://localhost:8080

Default Grafana Credentials

Username	Password
admin	admin

Dashboard ID

```
Node Exporter Full Dashboard ID: 1860
```